

■ PLAYWRIGHT

AUTOMATION INTERVIEW GUIDE

Complete Preparation: Basic → Advanced



Name : Rahul Dange
Role : Senior Lead Quality Engineer

Best of Luck with your Interviews!

16+
Sections

80+
Q&A

30+
Code
Samples

100%
Interview
Ready

■ Table of Contents

1	Fundamentals of Playwright	Architecture, Why Playwright, Browsers & Languages
2	Setup & Configuration	Installation, Project Structure, Config Deep-Dive
3	Core Concepts	Browser/Context/Page, Locators, Auto-wait, Assertions
4	Commands & Cheat Sheet	Navigation, Actions, Waits, Files, iFrames, Screenshots
5	Advanced Concepts	Parallel, Fixtures, POM, Data-driven, Tags, Retries
6	API Testing	APIRequestContext, CRUD, Schema Validation
7	Network Handling	Mocking, Intercepting, Slow Networks
8	CI/CD Integration	GitHub Actions, Jenkins, Allure, Reporting
9	Real-Time Scenarios	Flaky Tests, Shadow DOM, Auth, CAPTCHA, Race Conditions
10	Performance & Optimization	Parallel Strategies, Smart Waits, Speed Tips
11	Security Testing	JWT, Headers, Unauthorized Access Checks
12	Mobile & Cross-Browser	Device Emulation, Cross-browser Strategies
13	Interview Q&A;	Beginner (20+) · Intermediate (30+) · Advanced (30+) · Scenario · Coding
14	Hands-On Coding	Login E2E, POM Framework, API+UI, Mocking
15	Cheat Sheet Summary	Quick-Revision One-Pager — 10 Categories
16	Bonus Section	Common Mistakes, Interview Tips, Resume Tips

SECTION 1 — Fundamentals of Playwright

1. Fundamentals of Playwright

1.1 What is Playwright?

Playwright is an open-source, end-to-end test automation framework developed by Microsoft. It provides a single API to automate Chromium, Firefox, and WebKit browsers — including headless execution. Released in 2020, it quickly became the go-to framework for modern web test automation due to its speed, reliability, and powerful built-in feature set.

Feature	Description
Cross-browser	Chromium, Firefox, WebKit from one unified API
Cross-platform	Windows, macOS, Linux — all supported
Cross-language	JS/TS, Python, Java, C#
Auto-waiting	Eliminates flaky explicit waits automatically
Network interception	Mock, modify, or intercept any network request
Tracing / Video	Built-in trace viewer and video recording

1.2 Architecture & How It Works

Playwright communicates with browsers via native protocols — CDP (Chrome DevTools Protocol) for Chromium, a custom protocol for Firefox, and WebKit's remote debugging protocol. Communication happens over WebSocket, giving it full browser control without HTTP round-trips like Selenium's WebDriver.

- **Browser Server:** Playwright launches a persistent browser server subprocess.
- **BrowserContext:** Isolated sessions (like incognito) — own cookies, localStorage.
- **Page:** A tab within a context. All UI actions target a Page object.
- **Frame:** Each Page has a main frame; iFrames are separate Frame objects.

■ *Flow: Test Code → Playwright Node.js Server → WebSocket → Browser Process*

1.3 Playwright vs Selenium vs Cypress

Feature	Playwright	Selenium	Cypress
Speed	Very Fast	Slow	Fast
Auto-wait	Built-in	Manual only	Built-in
Multi-browser	All 3 (Cr/FF/WK)	All browsers	Limited
Multi-language	4 languages	Many languages	JS only
Network mock	Full native	Partial	Good
iFrame support	Easy	Complex	Limited
Multi-tab	Native support	Workaround needed	Very tricky
API Testing	Built-in	None	Plugin only
Parallel exec	Native workers	Selenium Grid	Paid plan

Shadow DOM	Supported	JS execute hack	Limited
Video/Trace	Native	None	Video only

1.4 Supported Languages & Browsers

Languages:

- JavaScript / TypeScript (primary)
- Python
- Java
- C# (.NET)

Browsers:

- Chromium (Chrome / Edge)
- Firefox
- WebKit (Safari)
- Mobile emulation (Android/iOS)

SECTION 2 — Setup & Configuration

■ ■ 2. Setup & Configuration

2.1 Installation (Step-by-Step)

#	Step	Command
1	Initialize Node.js project	<code>npm init -y</code>
2	Install Playwright (full)	<code>npm init playwright@latest</code>
3	Or manual install	<code>npm install -D @playwright/test</code>
4	Install all browsers	<code>npx playwright install</code>
5	Install specific browser	<code>npx playwright install chromium</code>
6	Run sample test	<code>npx playwright test</code>
7	View HTML report	<code>npx playwright show-report</code>

2.2 Project Structure

```
my-playwright-project/  
■ ■ ■ tests/ # Test files (.spec.ts)  
■ ■ ■ login.spec.ts  
■ ■ ■ dashboard.spec.ts  
■ ■ ■ api/  
■ ■ ■ user-api.spec.ts  
■ ■ ■ pages/ # Page Object Models  
■ ■ ■ LoginPage.ts  
■ ■ ■ DashboardPage.ts  
■ ■ ■ fixtures/ # Custom fixtures  
■ ■ ■ customFixtures.ts  
■ ■ ■ utils/ # Helper utilities  
■ ■ ■ testData.ts  
■ ■ ■ helpers.ts  
■ ■ ■ test-data/ # JSON / CSV data files  
■ ■ ■ playwright.config.ts # Main Playwright config  
■ ■ ■ .env # Environment variables  
■ ■ ■ package.json  
■ ■ ■ tsconfig.json
```

2.3 playwright.config.ts — Deep Explanation

```
import { defineConfig, devices } from '@playwright/test';  
import dotenv from 'dotenv';  
dotenv.config();
```

```
export default defineConfig({
  // --- Test discovery ---
  testDir: './tests',
  testMatch: '**/*.spec.ts',

  // --- Parallelism ---
  fullyParallel: true, // all tests run in parallel
  workers: process.env.CI ? 2 : 4, // 2 on CI, 4 locally

  // --- Retries ---
  retries: process.env.CI ? 2 : 0,

  // --- Reporters ---
  reporter: [
    ['html', { outputFolder: 'playwright-report' }],
    ['json', { outputFile: 'results.json' }],
    ['list'],
  ],

  // --- Global options ---
  use: {
    baseURL: process.env.BASE_URL || 'https://example.com',
    headless: true,
    viewport: { width: 1280, height: 720 },
    actionTimeout: 10_000,
    navigationTimeout: 30_000,
    screenshot: 'only-on-failure',
    video: 'on-first-retry',
    trace: 'on-first-retry',
    ignoreHTTPSErrors: true,
    locale: 'en-US',
    timezoneId: 'Asia/Kolkata',
  },

  // --- Projects (browsers) ---
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'Mobile Chrome', use: { ...devices['Pixel 5'] } },
    { name: 'setup', testMatch: /\.*\.\setup\.ts/ },
    { name: 'e2e', dependencies: ['setup'],
    use: { storageState: 'playwright/.auth/user.json' } },
  ],

  outputDir: 'test-results/',
  globalSetup: './global-setup.ts',
  globalTeardown: './global-teardown.ts',
});
```

2.4 Environment Setup

```
# .env file
BASE_URL=https://staging.myapp.com
API_URL=https://api.staging.myapp.com
ADMIN_USER=admin@test.com
ADMIN_PASS=SecurePass123
AUTH_TOKEN=eyJhbGciOiJIUzI1NiJ9...

// Accessing env vars in tests
import { test } from '@playwright/test';

test('env example', async ({ page }) => {
  // baseURL auto-used when set in config
  await page.goto('/login'); // resolves to baseURL + /login
  console.log(process.env.ADMIN_USER);
});
```

SECTION 3 — Core Concepts

3. Core Concepts

3.1 Browser, Context, Page

```
import { chromium } from '@playwright/test';

const browser = await chromium.launch({ headless: false });

// BrowserContext = isolated session (own cookies, storage)
const context = await browser.newContext({
  viewport: { width: 1920, height: 1080 },
  storageState: 'auth.json',
  httpCredentials: { username: 'user', password: 'pass' },
  geolocation: { latitude: 18.52, longitude: 73.85 },
  permissions: ['geolocation'],
});

// Page = a browser tab
const page = await context.newPage();
await page.goto('https://example.com');

// Multiple contexts = multiple users simultaneously
const adminCtx = await browser.newContext();
const userCtx = await browser.newContext();
```

■ **Best Practice:** Reuse context across tests in the same spec. Create a new page per test.

3.2 Locators — Best Practices & Anti-Patterns

Recommended Locators (Priority Order)

Priority	Locator Method	Example
1st	getByRole	page.getByRole('button', {name: 'Submit'})
2nd	getByLabel	page.getByLabel('Email')
3rd	getByPlaceholder	page.getByPlaceholder('Enter email')
4th	getByText	page.getByText('Welcome back')
5th	getByTestId	page.getByTestId('login-btn')
6th	getByTitle	page.getByTitle('Close dialog')
7th	locator (CSS/XPath)	page.locator('#submit-btn')

Anti-Patterns to Avoid

- **Absolute XPath:** //html/body/div[1]/form — breaks on any DOM change
- **Index-based selectors:** .item:nth-child(3) — fragile, order-dependent
- **Auto-generated IDs:** #ember123 — changes every build
- **CSS position selectors:** div > span:first-child — tightly coupled
- **Dynamic text matching:** getByText('User ID: 12345') — data-dependent

Chaining & Filtering Locators

```
// Filter within a parent element
const row = page.locator('tr').filter({ hasText: 'John Doe' });
await row.getByRole('button', { name: 'Edit' }).click();

// nth, first, last
await page.locator('.card').nth(2).click();
await page.locator('li').first().click();
await page.locator('li').last().click();

// and() - combine two locators
const btn = page.getByRole('button').and(page.getByText('Submit'));

// or() - match either locator
const el = page.getByLabel('Name').or(page.getByPlaceholder('Full name'));

// all() - get all matching elements as array
const items = await page.locator('.item').all();
for (const item of items) { await item.click(); }
```

3.3 Auto-Waiting Mechanism

Playwright automatically waits for elements to be actionable before performing any action. This eliminates 90% of explicit wait needs and dramatically reduces flakiness.

Action	Auto-waits for...
click()	Visible, enabled, stable, not obscured by other elements
fill()	Visible, enabled, editable
check()	Visible, enabled, currently unchecked
selectOption()	Visible, enabled select element
hover()	Visible, stable (animation done)
type()	Visible, enabled, editable
waitForURL()	Navigation to match the pattern
expect(locator)	Condition true — retries assertion until timeout

3.4 Assertions and expect() API

```
import { expect } from '@playwright/test';

// --- Locator assertions (auto-retrying) ---
await expect(page.getByRole('heading')).toBeVisible();
await expect(page.locator('#msg')).toHaveText('Success');
await expect(page.locator('#msg')).toContainText('Success');
await expect(page.locator('input')).toHaveValue('John');
await expect(page.locator('input')).toBeEnabled();
await expect(page.locator('input')).toBeDisabled();
await expect(page.locator('.loader')).toBeHidden();
await expect(page.locator('img')).toHaveAttribute('alt', 'Logo');
```

```
await expect(page.locator('.active')).toHaveClass(/active/);
await expect(page.locator('ul li')).toHaveLength(5);

// --- Page assertions ---
await expect(page).toHaveURL('https://example.com/dashboard');
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle('Dashboard | MyApp');

// --- Soft assertions (test continues even if these fail) ---
await expect.soft(page.locator('#price')).toHaveText('$100');
await expect.soft(page.locator('#qty')).toHaveText('2');

// --- Custom timeout ---
await expect(page.locator('#slow-element')).toBeVisible({ timeout: 15000 });
```

SECTION 4 — Commands & Cheat Sheet

■ 4. Commands & Cheat Sheet

4.1 Navigation

```
await page.goto('https://example.com');
await page.goto('/login'); // uses baseURL from config
await page.goto(url, { waitUntil: 'networkidle' });
await page.reload();
await page.goBack();
await page.goForward();
await page.waitForURL('**/dashboard');
await page.waitForURL(/dashboard/, { timeout: 5000 });
const url = page.url();
const title = await page.title();
```

4.2 Element Actions

```
// --- Click variants ---
await page.locator('#btn').click();
await page.locator('#btn').click({ button: 'right' });
await page.locator('#btn').dblclick();
await page.locator('#btn').click({ modifiers: ['Shift'] });
await page.locator('#btn').click({ force: true }); // bypass checks

// --- Input ---
await page.locator('#email').fill('test@test.com');
await page.locator('#email').clear();
await page.locator('#search').type('hello', { delay: 50 }); // human-like
await page.locator('#search').pressSequentially('hello');

// --- Dropdown ---
await page.locator('#country').selectOption('india');
await page.locator('#country').selectOption({ label: 'India' });
await page.locator('#tags').selectOption(['opt1', 'opt2']); // multi-select

// --- Checkbox / Radio ---
await page.locator('#terms').check();
await page.locator('#terms').uncheck();
await expect(page.locator('#terms')).toBeChecked();

// --- Get text / attribute ---
const text = await page.locator('h1').textContent();
const val = await page.locator('input').inputValue();
const attr = await page.locator('img').getAttribute('src');
const html = await page.locator('#block').innerHTML();
```

4.3 Waits

```
// Explicit waits
await page.waitForSelector('.spinner', { state: 'hidden' });
await page.waitForLoadState('networkidle');
await page.waitForLoadState('domcontentloaded');
await page.waitForFunction(() => window.myFlag === true);
await page.waitForTimeout(2000); // Avoid – use only as last resort

// Wait for network response after a click
const [resp] = await Promise.all([
  page.waitForResponse('**/api/users'),
  page.locator('#load').click(),
]);
const body = await resp.json();

// Wait for navigation
await Promise.all([
  page.waitForNavigation(),
  page.locator('#submit').click(),
]);
```

4.4 Keyboard & Mouse Actions

```
// --- Keyboard ---
await page.keyboard.press('Enter');
await page.keyboard.press('Control+A');
await page.keyboard.press('Tab');
await page.keyboard.type('Hello World');
await page.keyboard.down('Shift');
await page.keyboard.up('Shift');

// --- Mouse ---
await page.mouse.click(100, 200);
await page.mouse.dblclick(100, 200);
await page.mouse.move(100, 200);
await page.mouse.down();
await page.mouse.up();

// --- Hover ---
await page.locator('#menu').hover();

// --- Drag and Drop ---
await page.locator('#source').dragTo(page.locator('#target'));

// Manual drag
await page.locator('#src').hover();
await page.mouse.down();
await page.locator('#tgt').hover();
await page.mouse.up();
```

4.5 File Upload & Download

```
// --- File Upload ---
await page.locator('input[type=file]').setInputFiles('path/to/file.pdf');
```

```
await page.locator('input[type=file]').setInputFiles(['file1.pdf', 'file2.pdf']);

// --- File Download ---
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.locator('#download-btn').click(),
]);
const path = await download.path();
await download.saveAs('./downloads/report.pdf');
console.log('Downloaded:', download.suggestedFilename());
```

4.6 Screenshots & Videos

```
// Screenshots
await page.screenshot({ path: 'screenshot.png' });
await page.screenshot({ path: 'full.png', fullPage: true });
await page.locator('#chart').screenshot({ path: 'chart.png' });

// Video – configure in playwright.config.ts
// use: { video: 'on' | 'off' | 'on-first-retry' | 'retain-on-failure' }

// Retrieve video path after test
const video = page.video();
if (video) await video.saveAs('./videos/test.webm');

// Trace recording
await context.tracing.start({ screenshots: true, snapshots: true });
// ... test actions ...
await context.tracing.stop({ path: 'trace.zip' });
// View: npx playwright show-trace trace.zip
```

4.7 iFrame Handling

```
// Access iframe by locator
const frame = page.frameLocator('#my-iframe');
await frame.locator('#submit').click();

// Access frame by name or URL
const frame2 = page.frame({ name: 'payment' });
const frame3 = page.frame({ url: '/stripe/' });

// Nested iframes
const nested = page
  .frameLocator('#outer-frame')
  .frameLocator('#inner-frame');
await nested.locator('input').fill('value');

// Get all frames
const allFrames = page.frames();
for (const f of allFrames) { console.log(f.url()); }
```

SECTION 5 — Advanced Concepts

■ 5. Advanced Concepts

5.1 Parallel Execution & Workers

```
// playwright.config.ts
export default defineConfig({
  fullyParallel: true, // all tests within files also parallel
  workers: '50%', // use 50% of CPU cores
  // workers: 4, // or fixed number
});

// Force serial execution for a specific describe block
test.describe.configure({ mode: 'serial' });

// Run only this file with 1 worker (serial)
// npx playwright test login.spec.ts --workers=1
```

5.2 Fixtures & Custom Fixtures

```
// fixtures/customFixtures.ts
import { test as base, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

type MyFixtures = {
  loginPage: LoginPage;
  loggedInPage: LoginPage;
};

export const test = base.extend<MyFixtures>({
  // Basic fixture
  loginPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await use(loginPage);
    // Optional teardown after test
    await page.close();
  },

  // Pre-authenticated fixture
  loggedInPage: async ({ page }, use) => {
    const lp = new LoginPage(page);
    await lp.navigate();
    await lp.login('admin@test.com', 'password123');
    await use(lp);
  },
});

export { expect };

// Usage in test file
```

```
import { test, expect } from '../fixtures/customFixtures';
test('dashboard loads', async ({ loggedInPage }) => {
  await expect(loggedInPage.page.locator('h1')).toHaveText('Dashboard');
});
```

5.3 Hooks

```
import { test, expect } from '@playwright/test';

test.describe('User Management', () => {
  test.beforeAll(async ({ browser }) => {
    // Runs ONCE before all tests – setup DB, create test data
  });

  test.afterAll(async () => {
    // Runs ONCE after all tests – cleanup
  });

  test.beforeEach(async ({ page }) => {
    await page.goto('/users');
    await page.waitForLoadState('networkidle');
  });

  test.afterEach(async ({ page }, testInfo) => {
    if (testInfo.status !== 'passed') {
      await page.screenshot({ path: `failures/${testInfo.title}.png` });
    }
  });

  test('list users', async ({ page }) => { /* ... */ });
  test('add user', async ({ page }) => { /* ... */ });
});
```

5.4 Page Object Model (POM)

```
// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly submitBtn: Locator;
  readonly errorMsg: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.submitBtn = page.getByRole('button', { name: 'Sign In' });
    this.errorMsg = page.locator('.error-message');
  }

  async navigate() {
```

```
await this.page.goto('/login');
}

async login(email: string, password: string) {
  await this.emailInput.fill(email);
  await this.passwordInput.fill(password);
  await this.submitBtn.click();
}

async expectError(msg: string) {
  await expect(this.errorMsg).toHaveText(msg);
}
}
```

5.5 Data-Driven Testing

```
const users = [
  { email: 'admin@test.com', pass: 'Admin123', role: 'Admin' },
  { email: 'user@test.com', pass: 'User123', role: 'User' },
  { email: 'viewer@test.com', pass: 'Viewer123', role: 'Viewer' },
];

for (const { email, pass, role } of users) {
  test(`Login as ${role}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(email);
    await page.getByLabel('Password').fill(pass);
    await page.getByRole('button', { name: 'Sign In' }).click();
    await expect(page.locator('.user-role')).toHaveText(role);
  });
}

// From external JSON file
import testData from '../test-data/users.json';
testData.forEach(({ email, pass, expected }) => {
  test(`login: ${email}`, async ({ page }) => { /* ... */ });
});
```

5.6 Retry Mechanism

```
// Global retries – in config
export default defineConfig({ retries: 2 });

// Per-test retry
test('flaky test', { retries: 3 }, async ({ page }) => { /* ... */ });

// Detect and handle retry in test body
test('retry-aware', async ({ page }, testInfo) => {
  if (testInfo.retry > 0) {
    await page.reload(); // Clear state on retry
  }
  // ... test logic ...
});
```

5.7 Test Tagging & Filtering

```
// Tag tests in the name
test('login works @smoke', async ({ page }) => { /* ... */ });

// Tag via annotation
test('full checkout', {
  tag: ['@regression', '@critical'],
}, async ({ page }) => { /* ... */ });

// Skip / Only / Fixme
test.skip('known issue', async () => { /* skipped */ });
test.only('focus this', async () => { /* runs only this */ });
test.fixme('bug #123', async () => { /* marks as fixme */ });

// CLI filtering
// npx playwright test --grep @smoke
// npx playwright test --grep-invert @slow
// npx playwright test --project=chromium
// npx playwright test tests/login.spec.ts
```

SECTION 6 — API Testing with Playwright

■ 6. API Testing with Playwright

6.1 APIRequestContext

```
import { test, expect } from '@playwright/test';

// Standalone API test (no browser needed)
test('GET users list', async ({ request }) => {
  const resp = await request.get('https://api.example.com/users');
  expect(resp.status()).toBe(200);
  const body = await resp.json();
  expect(body.users).toHaveLength(10);
});

// Custom APIRequestContext with auth headers
const apiContext = await request.newContext({
  baseURL: 'https://api.example.com',
  extraHTTPHeaders: {
    'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,
    'Content-Type': 'application/json',
  },
});
```

6.2 CRUD Operations

```
// GET
const getResp = await request.get('/api/users/1');
expect(getResp.ok()).toBeTruthy();
const user = await getResp.json();
expect(user.name).toBe('John Doe');

// POST
const postResp = await request.post('/api/users', {
  data: { name: 'Jane', email: 'jane@test.com', role: 'user' },
});
expect(postResp.status()).toBe(201);
const created = await postResp.json();
expect(created.id).toBeDefined();

// PUT
const putResp = await request.put(`/api/users/${created.id}`, {
  data: { name: 'Jane Updated' },
});
expect(putResp.status()).toBe(200);

// PATCH
const patchResp = await request.patch(`/api/users/${created.id}`, {
  data: { active: false },
```

```
});  
  
// DELETE  
const delResp = await request.delete(`/api/users/${created.id}`);  
expect(delResp.status()).toBe(204);
```

6.3 Schema Validation with AJV

```
import Ajv from 'ajv';  
const ajv = new Ajv();  
  
const userSchema = {  
  type: 'object',  
  required: ['id', 'name', 'email'],  
  properties: {  
    id: { type: 'number' },  
    name: { type: 'string' },  
    email: { type: 'string', format: 'email' },  
    role: { type: 'string', enum: ['admin', 'user', 'viewer' ] },  
  },  
  additionalProperties: false,  
};  
  
test('validate user schema', async ({ request }) => {  
  const resp = await request.get('/api/users/1');  
  const body = await resp.json();  
  const valid = ajv.compile(userSchema)(body);  
  expect(valid).toBe(true);  
});
```

SECTION 7 — Network Handling

■ 7. Network Handling

7.1 Mocking APIs

```
// Mock a GET response
await page.route('**/api/users', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: 1, name: 'Mock User', role: 'admin' },
    ]),
  });
});

// Mock an error
await page.route('**/api/orders', (route) =>
  route.fulfill({ status: 500, body: 'Internal Server Error' })
);

// Mock from a JSON fixture file
await page.route('**/api/products', (route) =>
  route.fulfill({ path: './mocks/products.json' })
);

// Remove a mock
await page.unroute('**/api/users');
```

7.2 Intercepting & Modifying Requests

```
// Intercept and log request details
await page.route('**/api/search', async (route) => {
  const request = route.request();
  console.log('URL:', request.url());
  console.log('Method:', request.method());

  // Modify headers and pass through
  await route.continue({
    headers: { ...request.headers(), 'x-custom': 'injected' },
  });
});

// Block images and analytics
await page.route('**/*analytics*', (r) => r.abort());
await page.route('**/*.{png,jpg,gif,webp}', (r) => r.abort());

// Listen to all API responses
page.on('response', (response) => {
  if (response.url().includes('/api/')) {
```

```
console.log(`${response.status()} ${response.url()}`);  
}  
});
```

7.3 Handling Slow Networks

```
// Emulate slow 3G via Chrome DevTools Protocol  
const cdpSession = await context.newCDPSession(page);  
await cdpSession.send('Network.enable');  
await cdpSession.send('Network.emulateNetworkConditions', {  
  offline: false,  
  downloadThroughput: 500 * 1024 / 8, // 500 kbps  
  uploadThroughput: 500 * 1024 / 8,  
  latency: 400, // 400ms latency  
});  
  
// Offline mode  
await page.context().setOffline(true);  
await page.context().setOffline(false); // back online
```

SECTION 8 — CI/CD Integration

■ 8. CI/CD Integration

8.1 GitHub Actions Workflow

```
# .github/workflows/playwright.yml
name: Playwright Tests

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'npm'
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright browsers
        run: npx playwright install --with-deps chromium
      - name: Run tests
        run: npx playwright test
    env:
      BASE_URL: ${ secrets.STAGING_URL }
      AUTH_TOKEN: ${ secrets.AUTH_TOKEN }
      - name: Upload HTML report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30
```

8.2 Allure Reporting Setup

```
# Install
npm install -D allure-playwright

// playwright.config.ts
```

```
reporter: [  
  ['line'],  
  ['allure-playwright', { outputFolder: 'allure-results' }]],  
],  
  
# Generate and open report locally  
npx allure generate allure-results --clean  
npx allure open
```

8.3 Test Sharding for CI Parallelism

```
# Distribute tests across 4 CI machines  
# Machine 1:  
npx playwright test --shard=1/4  
# Machine 2:  
npx playwright test --shard=2/4  
# Machine 3:  
npx playwright test --shard=3/4  
# Machine 4:  
npx playwright test --shard=4/4  
  
# Merge blob reports from all shards  
npx playwright merge-reports ./all-blobs --reporter html
```

SECTION 9 — Real-Time Scenarios & Edge Cases

■ 9. Real-Time Scenarios & Edge Cases

9.1 Handling Flaky Tests

- **Root cause:** Timing issues, dynamic content, network latency, animations
- **Fix 1:** Replace `waitForTimeout()` with `expect().toBeVisible()`
- **Fix 2:** Use `page.waitForLoadState('networkidle')` after navigation
- **Fix 3:** Set retries: 2 in config for CI
- **Fix 4:** Use `Promise.all` to sync click with network response

```
// Bad - flaky
await page.waitForTimeout(3000);
await page.click('#submit');

// Good - stable
await expect(page.locator('#submit')).toBeEnabled();
await page.locator('#submit').click();
```

9.2 Dynamic Elements

```
// Use data-testid instead of fragile IDs
await page.locator('[data-testid="submit-btn"]').click();

// Wait for loading to disappear
await expect(page.locator('.loading-spinner')).toBeHidden();
await page.locator('.actual-content').click();

// Find table row by dynamic text
const row = page.locator('tr').filter({ hasText: 'John Doe' });
await row.getByRole('button', { name: 'Edit' }).click();
```

9.3 Shadow DOM

```
// Playwright auto-pierces Shadow DOM with CSS selectors
await page.locator('my-component >> input').fill('value');

// Using evaluate for complex Shadow DOM
const shadowInput = await page.evaluateHandle(() => {
  const host = document.querySelector('my-element');
  return host.shadowRoot.querySelector('input');
});
await shadowInput.asElement().fill('value');
```

9.4 Multiple Tabs / Windows

```
// Handle new tab opened by a click
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.locator('#open-tab-btn').click(),
]);
```

```
]);
await newPage.waitForLoadState();
await expect(newPage).toHaveURL(/expected-page/);
await newPage.close();

// Switch between tabs
const pages = context.pages();
await pages[1].bringToFront();
```

9.5 Authentication Handling (Best Practice)

```
// auth.setup.ts – runs ONCE, saves auth state
import { test as setup } from '@playwright/test';

setup('authenticate', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('admin@test.com');
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Sign In' }).click();
  await page.waitForURL('/dashboard');

  // Save entire session state
  await page.context().storageState({
    path: 'playwright/.auth/admin.json',
  });
});

// playwright.config.ts – load auth for all tests
projects: [
  { name: 'setup', testMatch: /auth.setup.ts/ },
  { name: 'e2e',
    dependencies: ['setup'],
    use: { storageState: 'playwright/.auth/admin.json' },
  },
],
```

9.6 CAPTCHA / OTP Strategies

- **Strategy 1 — Disable in test env:** Add a flag/header to skip CAPTCHA for test environments (most common approach).
- **Strategy 2 — Mock CAPTCHA API:** Intercept the verification endpoint and return success.
- **Strategy 3 — Test CAPTCHA keys:** Google reCAPTCHA provides test site keys that always pass.
- **Strategy 4 — OTP via test inbox:** Use Mailhog or Mailinator API to fetch OTP from test email.

```
// Mock CAPTCHA verification endpoint
await page.route('**/recaptcha/api/verify', (route) =>
  route.fulfill({ status: 200, body: JSON.stringify({ success: true }) })
);
```

9.7 Race Conditions

```
// Sync click with expected API call
const [response] = await Promise.all([
```

```
page.waitForResponse('**/api/submit'),
page.locator('#submit').click(),
]);

// Handle two possible outcomes
const success = page.locator('.success-msg');
const error = page.locator('.error-msg');
await expect(success.or(error)).toBeVisible();

// Wait for specific element with retry
await page.locator('#result').waitFor({ state: 'visible' });
```

SECTION 10 — Performance & Optimization

■ 10. Performance & Optimization

10.1 Speed Improvement Strategies

- **Parallel workers:** fullyParallel: true + workers based on CPU cores
- **Reuse auth state:** storageState avoids repeated login (saves 3-5s per test)
- **Block media:** Abort image/font requests in functional tests
- **API for data setup:** Create test data via API, not UI — 10x faster
- **Headless mode:** Always use headless: true in CI
- **Smart waits:** Never use waitForTimeout — use assertions and event waits

```
// Block images & fonts to speed up tests
await page.route('**/*.{png,jpg,jpeg,gif,webp,svg,woff,woff2}',
(route) => route.abort()
);

// Create test data via API (not UI)
const resp = await request.post('/api/users', {
  data: { name: 'Test User', email: 'test@test.com' },
});
const { id } = await resp.json();
// Now test the UI that shows this user
await page.goto(`/users/${id}`);
```

10.2 Smart Waits vs Hard Waits

Type	Example	Verdict
Smart	expect(locator).toBeVisible()	Always use
Smart	page.waitForURL(/dashboard/)	Always use
Smart	page.waitForResponse('**/api')	Always use
Smart	page.waitForLoadState('networkidle')	Use when needed
OK	page.waitForSelector('.element')	Acceptable
Avoid	page.waitForTimeout(3000)	Never — causes flakiness

SECTION 11 — Security Testing Basics

■ 11. Security Testing Basics

11.1 JWT Token Validation

```
test('JWT token in login response', async ({ request }) => {
  const resp = await request.post('/api/auth/login', {
    data: { email: 'user@test.com', password: 'pass' }
  });
```

```
});  
const body = await resp.json();  
expect(body.token).toBeDefined();  
  
// Decode and validate JWT payload  
const [, payload] = body.token.split('.');  
const decoded = JSON.parse(atob(payload));  
expect(decoded.exp).toBeGreaterThan(Date.now() / 1000);  
expect(decoded.role).toBe('user');  
});
```

11.2 Security Headers & Unauthorized Access

```
test('security headers present', async ({ request }) => {  
  const resp = await request.get('/');  
  const headers = resp.headers();  
  expect(headers['x-content-type-options']).toBe('nosniff');  
  expect(headers['x-frame-options']).toMatch(/DENY|SAMEORIGIN/);  
  expect(headers['strict-transport-security']).toBeDefined();  
  expect(headers['x-powered-by']).toBeUndefined(); // should be hidden  
});  
  
test('401 without auth token', async ({ request }) => {  
  const resp = await request.get('/api/admin/users');  
  expect(resp.status()).toBe(401);  
});  
  
test('403 on privilege escalation', async ({ request }) => {  
  const login = await request.post('/api/auth/login', {  
    data: { email: 'user@test.com', password: 'pass' }  
  });  
  const { token } = await login.json();  
  const resp = await request.get('/api/admin/users', {  
    headers: { Authorization: `Bearer ${token}` }  
  });  
  expect(resp.status()).toBe(403);  
});
```

SECTION 12 — Mobile & Cross-Browser Testing

12. Mobile & Cross-Browser Testing

12.1 Device Emulation

```
import { devices } from '@playwright/test';  
  
// In playwright.config.ts  
projects: [  
  { name: 'Desktop Chrome', use: { ...devices['Desktop Chrome'] } },  
  { name: 'iPhone 14', use: { ...devices['iPhone 14'] } },
```

```
{ name: 'Pixel 7', use: { ...devices['Pixel 7'] } },
{ name: 'iPad Pro', use: { ...devices['iPad Pro 11'] } },
],

// Custom emulation in a test
const context = await browser.newContext({
  ...devices['iPhone 14'],
  geolocation: { latitude: 18.52, longitude: 73.85 },
  permissions: ['geolocation'],
  colorScheme: 'dark',
});

// Test different viewport
await page.setViewportSize({ width: 375, height: 812 }); // mobile
await page.setViewportSize({ width: 1440, height: 900 }); // desktop
```

SECTION 13 — Interview Questions & Answers

■ 13. Interview Questions & Answers

Beginner Level (20 Questions)

Q: What is Playwright?

An open-source, end-to-end automation framework by Microsoft for Chromium, Firefox, and WebKit using a single API. Built for speed and reliability with auto-waiting, tracing, and video recording built-in.

Q: What languages does Playwright support?

JavaScript, TypeScript (primary), Python, Java, and C# (.NET). TypeScript has the most features and community support.

Q: What browsers does Playwright support?

Chromium (Chrome/Edge), Firefox, WebKit (Safari). Mobile emulation via device profiles like 'iPhone 14' or 'Pixel 7'.

Q: How is Playwright different from Selenium?

Playwright is faster (native protocol), has built-in auto-waiting, native multi-tab/iframe support, built-in API testing, trace viewer, and no separate driver management. Selenium requires WebDriver + external libraries for many features.

Q: What is a BrowserContext?

An isolated browser session like incognito mode — has its own cookies, localStorage, sessionStorage. Multiple contexts can run simultaneously. Use one context per user/role in tests.

Q: What is the difference between `page.click()` and `locator.click()`?

`page.click(selector)` is legacy shorthand. `locator.click()` uses the modern Locator API which is reusable, composable, and retry-aware. Always prefer locator-based actions.

Q: How does auto-waiting work?

Before any action (click, fill, etc.), Playwright waits for the element to be visible, stable (not animating), enabled, and not obscured. Eliminates most explicit wait needs.

Q: What is the default timeout?

30 seconds for most actions. Configurable globally with `actionTimeout` in config, per-test with `test.setTimeout()`, or per-action with `{ timeout: 5000 }`.

Q: How do you take a screenshot?

`await page.screenshot({ path: 'screenshot.png' });` Full page: add `fullPage: true`. Element-only: `locator.screenshot({ path: '...' })`.

Q: How do you fill a form field?

`await page.getByLabel('Email').fill('test@example.com');` — clears and fills. Use `type()` to simulate keystroke-by-keystroke for character-by-character input.

Q: Why is `getByRole()` preferred for locators?

It uses ARIA roles and accessible names, making tests resilient to HTML changes and also validating accessibility. It's the most stable and semantically meaningful locator.

Q: How do you assert a page title?

`await expect(page).toHaveTitle('My App');` or: `const title = await page.title(); expect(title).toContain('App');`

Q: What is `test.beforeEach()`?

A hook that runs before every test in its describe block. Use for common navigation, state setup, or data preparation shared across tests in the group.

Q: How do you run Playwright tests?

`npx playwright test` — all. `npx playwright test login.spec.ts` — specific file. `--headed` for visible browser. `--debug` for step-by-step. `--grep @smoke` for tags.

Q: What is headless mode?

Browser runs without a GUI — faster and ideal for CI/CD. Default in Playwright. Use `--headed` flag or `headless: false` in config for visible browser during local debugging.

Q: How do you handle dropdowns?

`await page.locator('select').selectOption('value');` by label: `selectOption({ label: 'India' });` For custom dropdowns (not native select), click to open, then click the option.

Q: How do you verify an element is visible?

`await expect(page.locator('#element')).toBeVisible();` — auto-retries until visible or times out. Most reliable assertion method.

Q: What does `fullyParallel` mean in config?

When true, individual tests within the same file also run in parallel (not just different files). Set to false to run tests within each file sequentially.

Q: How do you configure `baseURL`?

Set use: `{ baseURL: 'https://example.com' }` in `playwright.config.ts`. Then use relative URLs in tests: `await page.goto('/login');` — auto-resolves to full URL.

Q: What is the Playwright test runner (`@playwright/test`)?

The built-in test framework with: full parallelism, fixtures, hooks, reporters, retries, and sharding. Separate from playwright core (playwright package).

Intermediate Level (30 Questions)

Q: What is POM and why use it?

Page Object Model is a design pattern where each page has a dedicated class with locators and actions. Benefits: reusability, maintainability, single place to update when UI changes, better readability.

Q: Explain fixtures in Playwright.

Fixtures are setup/teardown functions that inject dependencies into tests. Built-in: page, browser, context, request. Custom fixtures extend these for app-specific needs like pre-authenticated pages or seeded test data.

Q: What is storageState() and when to use it?

Saves cookies + localStorage + sessionStorage as JSON. Use to reuse authentication across tests — login once in a setup project, save state, load it in all tests. Avoids repeated login (saves seconds per test).

Q: How do you handle file downloads?

Wrap trigger and event listener in Promise.all: `const [dl] = await Promise.all([ page.waitForEvent('download'), page.locator('#dl-btn').click() ]);` Then `dl.path()` or `dl.saveAs('./path')`.

Q: What is page.evaluate()?

Executes JavaScript in the browser context. Use for: reading DOM values, setting window variables, manipulating storage, scrolling to coordinates, or accessing app-specific JS state.

Q: Soft vs hard assertions?

Hard assertions (expect) stop the test immediately on first failure. Soft assertions (expect.soft) record failures but continue — allowing you to collect multiple failures in one test run.

Q: How to handle auth for large test suites?

Create a setup project (auth.setup.ts) that logs in and saves storageState. All test projects list it as a dependency and load the saved JSON. Login runs once for the entire suite.

Q: What is tracing?

Playwright records every action, DOM snapshot, and network request as a trace. Enable via trace: 'on-first-retry'. View with: `npx playwright show-trace trace.zip`. Essential for debugging CI failures.

Q: How do you test iframes?

Use `page.frameLocator('#iframe-id')` for a scoped locator. Then use normal locators within it. Chain `.frameLocator()` for nested iframes. Can also use `page.frame({ name: 'frameName' })`.

Q: What are test tags?

Labels added to test names (@smoke, @regression) or via tag: `['@smoke']` annotation. Run filtered: `--grep @smoke`. Exclude: `--grep-invert @slow`. Useful for running subsets in CI.

Q: Explain retry mechanism.

Set retries: 2 in config. Playwright re-runs failed tests. `testInfo.retry` gives the current attempt number. Can use it to clear state on retry. Video/trace only captured on first retry by default.

Q: How to handle popups/dialogs?

Register handler BEFORE triggering it: `page.on('dialog', d => d.accept())`. For one-time: `page.once('dialog', d => d.fill('text'))`. Handles alert, confirm, prompt dialogs.

Q: What is APIRequestContext?

Built-in HTTP client for API testing. Available via the request fixture. Use standalone (no browser) or alongside UI tests for data setup/teardown. Much faster than UI-based data creation.

Q: Wait for network response after click?

`Promise.all: const [resp] = await Promise.all([ page.waitForResponse('**/api/data'), page.locator('#btn').click() ]);` Both start simultaneously — ensures they're concurrent.

Q: page.waitForSelector() vs expect().toBeVisible()?

`waitForSelector` returns legacy `ElementHandle` (not retry-aware). `expect().toBeVisible()` uses modern `Locator` API with built-in retry and integrates with test reporters. Always prefer the latter.

Q: How do you test multiple browser tabs?

`const [newPage] = await Promise.all([ context.waitForEvent('page'), page.click('#open-tab') ])`; Then use `newPage` as a normal `Page` object.

Q: How to run specific browser projects?

`npx playwright test --project=chromium` — one project. `npx playwright test --project=chromium --project=firefox` — multiple. Defined in `projects` array in config.

Q: What is `test.describe()`?

Groups related tests. Enables shared hooks (`beforeEach/afterAll`) scoped to the group. Supports nesting. Use `.configure({ mode: 'serial' })` for sequential execution within the group.

Q: How do you debug failing tests?

1) `npx playwright test --debug` (opens inspector), 2) `await page.pause()` in test, 3) `npx playwright codegen` for recording, 4) `npx playwright show-trace` for CI failures, 5) `PWDEBUG=1` env var.

Q: What is the workers option?

Controls parallel worker count. Can be a number (`workers: 4`) or percentage of CPUs (`workers: '50%'`). On CI, limit to 2-4 to avoid resource contention.

Q: How do you mock an API?

`await page.route(urlPattern, handler)`. Handler can: `fulfill()` with custom response, `continue()` to pass modified request, `abort()` to block. Remove with `page.unroute()`.

Q: What is Playwright Codegen?

`npx playwright codegen url` — records browser actions and generates Playwright code. Good starting point, but generated code needs refactoring to use proper locators and POM patterns.

Q: How to handle hover-triggered menus?

`await page.locator('#nav-item').hover();` `await expect(page.locator('#dropdown')).toBeVisible();` `await page.locator('#dropdown-link').click();`

Q: How to validate an API schema?

Use `AJV` library: define a JSON schema, compile it, run against response body. Playwright has no built-in schema validation — integrate `AJV` or `Zod`.

Q: How to test date pickers?

Method 1: Direct fill if input accepts text. Method 2: `page.evaluate()` to set value + dispatch change event. Method 3: Click through the calendar UI.

Q: What is `expect.poll()`?

Retries a custom function until it returns the expected value: `await expect.poll(async () => (await getStatus()), { timeout: 30000 }).toBe('complete');`

Q: How do you handle cookies?

Get: `await context.cookies();` Set: `await context.addCookies([ {name, value, url} ])`; Clear: `await context.clearCookies();` Persist: use `storageState`.

Q: Parallelism vs concurrency in Playwright?

Parallelism: multiple worker processes run simultaneously, each with own browser/context/page. Concurrency: `async/await` within one worker. Playwright's parallelism is process-level.

Q: How do you manage multiple environments?

Use `.env` files and `dotenv`. Switch by `ENV=staging npx playwright test`. Read in config via `process.env.BASE_URL`. Never hardcode URLs in test files.

Q: What is `locator.filter()`?

Narrows a broad locator: `page.locator('tr').filter({ hasText: 'Active' })` — only rows with 'Active'. `filter({ has: locator })` — rows containing a specific child element.

Advanced Level (30 Questions)

Q: Explain Playwright's architecture.

Uses CDP for Chromium, custom protocol for Firefox, WebKit's remote inspection protocol. Communicates via WebSocket. Launches browsers as subprocesses. Can run remote browser servers (playwright.connect). No HTTP round-trips like Selenium WebDriver.

Q: How to implement a custom reporter?

Implement the Reporter interface: `class MyReporter { onBegin(config,suite){} onTestBegin(test){} onTestEnd(test,result){} onEnd(result){} }`. Register: `reporter: ['./my-reporter.ts']` in config.

Q: What is globalSetup/globalTeardown?

`globalSetup` runs ONCE before all workers start. Used for: starting test server, seeding DB, creating auth tokens. `globalTeardown` runs once after all workers complete. Configure in `playwright.config.ts`.

Q: Multi-role authentication strategy?

Create separate setup specs per role (`admin.setup.ts`, `user.setup.ts`). Save to separate state files. Create separate projects with dependencies on respective setups and matching `storageState` files.

Q: Explain fixture dependency injection.

Playwright resolves fixture dependencies automatically. Each fixture declares what it needs (page, browser) and what it provides. Framework builds the dependency graph and instantiates in correct order. Same-scope fixtures are shared.

Q: How to handle WebSockets?

`page.on('websocket', ws => { ws.on('framesent', f => {}); ws.on('framereceived', f => {}); })`. Use `page.waitForEvent('websocket')` to get the WebSocket. Can inspect but not easily mock — use server-side mocking instead.

Q: route.fulfill() vs route.continue()?

`fulfill()` returns custom response (mocking). `continue()` forwards to server with optional modifications to URL/headers/body. `abort()` cancels the request entirely.

Q: How to implement custom retry with backoff?

Playwright's retries are simple re-runs. For backoff inside a test: `async function retry(fn, n=3){ try{ return await fn(); } catch(e){ if(n>0){ await page.waitForTimeout(1000); return retry(fn,n-1); } throw e; } }`

Q: What is page.addInitScript()?

Runs a script in EVERY page/frame BEFORE any other script. Used to: mock `Date/Math.random`, inject polyfills, set window globals. Persists across navigations.

Q: How to test service workers?

`context.on('serviceworker', sw => {})` to listen. `context.serviceWorkers()` to get all. `await context.waitForEvent('serviceworker')` to wait for registration.

Q: Explain test sharding.

Splits suite across CI machines: `--shard=1/4, 2/4`, etc. Each machine runs its portion. Combine with: `npx playwright merge-reports ./blobs --reporter html`. Dramatically reduces total CI time.

Q: Handling flaky network calls?

1) Mock the unreliable API. 2) Set retries. 3) Use `expect.poll` with timeout. 4) Add `waitForResponse` to ensure call completes. 5) Set appropriate `actionTimeout`. 6) `waitForLoadState('networkidle')`.

Q: What is Playwright Component Testing?

`@playwright/experimental-ct-react` mounts individual React components in isolation for testing without a full server. Uses `mount()` fixture. Similar to Cypress Component Testing.

Q: How to test drag and drop?

Method 1: `locator.dragTo(targetLocator)`. Method 2: Manual mouse events — `hover` source, `mousedown`, `move` to target, `mouseup`. Method 3: `page.evaluate()` with `DataTransfer` for HTML5 drag APIs.

Q: toHaveText() vs toContainText()?

toHaveText() matches the FULL text content (with whitespace normalization). toContainText() checks the text CONTAINS the given string. Use toContainText for partial matching.

Q: What is locator.filter() and when to use it?

Narrows a locator set: page.locator('tr').filter({ hasText: 'Active' }). filter({ has: otherLocator }) finds elements containing a specific child. Useful for table rows, list items, and card components.

Q: How to test canvas elements?

Canvas is not accessible via DOM. Options: 1) page.evaluate() to call canvas API. 2) expect(locator).toHaveScreenshot() for visual comparison. 3) Test the underlying data model if app exposes it.

Q: What is the trace viewer?

npx playwright show-trace trace.zip shows: full timeline, before/after DOM snapshots, network log, console, test steps. Invaluable for debugging CI failures without needing to re-run.

Q: Connecting to a remote browser?

const browser = await chromium.connect('ws://remote-host:9222'); Or use playwrights own server: const server = await chromium.launchServer(); await chromium.connect(server.wsEndpoint());

Q: What is expect(locator).toHaveScreenshot()?

Visual regression testing. First run creates baseline. Subsequent runs compare against baseline with configurable pixel tolerance. Update baselines with --update-snapshots flag.

Q: Handling HTTPS certificate errors?

Set ignoreHTTPSErrors: true in context options or config use section. Applied globally — useful for self-signed certs in staging/test environments.

Q: What is test.step()?

Groups actions into named steps: await test.step('Login as admin', async()=>{ ... }). Steps appear in HTML report and trace viewer as collapsible sections — makes long tests readable.

Q: How to implement polling?

await expect.poll(async () => { const r = await request.get('/status'); return (await r.json()).status; }, { timeout: 30000, intervals: [1000, 2000, 5000] }).toBe('complete');

Q: What is testInfo?

Runtime test metadata: testInfo.title, testInfo.retry, testInfo.status, testInfo.outputDir. Available in test body and afterEach. Use to conditionally handle retries or attach artifacts.

Q: How to attach files to reports?

await testInfo.attach('screenshot', { contentType: 'image/png', body: buffer }); or await testInfo.attach('log', { path: './log.txt', contentType: 'text/plain' }); Appears in HTML report.

Q: What is HAR format in Playwright?

HTTP Archive — records all network traffic. Use await context.routeFromHAR('./rec.har') to replay recorded traffic. Enables stable offline testing or deterministic network mocking.

Q: How to ensure test isolation?

1) Fresh BrowserContext per test (default). 2) API teardown in afterEach. 3) storageState for auth. 4) No test order dependencies. 5) Don't share mutable state between tests.

Q: What is locator.all()?

Returns array of all matching locators: const items = await page.locator('.item').all(); for(const item of items){ await expect(item).toBeVisible(); }. Use when iterating over all matching elements.

Q: How to test responsive layouts?

page.setViewportSize({ width, height }) at breakpoints. Or configure device profiles. Combine with visual screenshots per breakpoint. Test navigation, layout, and functionality at mobile/tablet/desktop.

Q: Parallelism best practices?

1) fullyParallel: true. 2) No shared mutable state. 3) API-based data setup. 4) storageState for auth. 5) Workers: '50%' locally, 2-4 on CI. 6) Use --shard for multi-machine CI.

Scenario-Based Questions

Q: Test fails on CI but passes locally. How do you debug?

1. Enable trace: 'on-first-retry' and download trace artifact from CI.
2. Run with `--workers=1` to rule out parallelism issues.
3. Increase `actionTimeout` — CI machines are often slower.
4. Verify browser version: `npx playwright install --with-deps` on CI.
5. Check env variables are correctly set in CI secrets.
6. Review if test has timing issues: replace all `waitForTimeout` with proper assertions.

Q: How do you test a login flow with 2FA/OTP?

1. Configure test environment to use a fixed OTP (e.g., 123456) — most practical.
2. Or fetch OTP from a test email inbox via Mailhog/Mailinator API.
3. Mock the OTP verification endpoint to always return success.
4. For SMS OTP: use Twilio test credentials with a virtual number.

Q: How would you automate a multi-step checkout?

1. Use `storageState` for pre-authenticated user — skip login.
2. Add items to cart via API (`POST /cart/add`) — skip UI setup.
3. Test UI steps: shipping address, payment info, confirmation.
4. Mock payment gateway to avoid real transactions.
5. Assert order confirmation number in final step.
6. Teardown: cancel order via API in `afterEach`.

Q: How do you handle tests that must run in a specific order?

Playwright tests should ideally be independent. If ordering is required:

1. Use `test.describe.configure({ mode: 'serial' })` in the describe block.
2. Use `beforeAll` to set up shared state for the group.
3. Pass state via variables declared in the describe scope.
4. Better: refactor to remove ordering dependency using fixtures and API setup.

Coding Questions

Q: Write a complete login test with valid and invalid scenarios

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test.describe('Login Tests', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
  });

  test('TC001 - Valid login redirects to dashboard', async ({ page }) => {
    const lp = new LoginPage(page);
    await lp.login('admin@test.com', 'Admin@123');
    await expect(page).toHaveURL(/dashboard/);
    await expect(page.getByRole('heading')).toContainText('Dashboard');
  });

  test('TC002 - Invalid password shows error message', async ({ page }) => {
    const lp = new LoginPage(page);
    await lp.login('admin@test.com', 'wrongpass');
    await expect(lp.errorMsg).toHaveText('Invalid credentials');
    await expect(page).toHaveURL('/login');
  });

  test('TC003 - Empty form shows validation errors', async ({ page }) => {
    await page.getByRole('button', { name: 'Sign In' }).click();
    await expect(page.locator('.email-error')).toHaveText('Email is required');
    await expect(page.locator('.pass-error')).toHaveText('Password is required');
  });
});
```

Q: Write a test that mocks an API and verifies UI renders mocked data

```
test('display mocked user list', async ({ page }) => {
  // Setup mock BEFORE navigating
  await page.route('**/api/users', async (route) => {
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({
        users: [
          { id: 1, name: 'Alice Mock', role: 'Admin' },
          { id: 2, name: 'Bob Mock', role: 'User' },
        ]
      })
    });
  });

  await page.goto('/users');
  await expect(page.locator('table tbody tr')).toHaveCount(2);
});
```

```
await expect(page.locator('td').first()).toHaveText('Alice Mock');  
});
```

SECTION 14 — Hands-On Coding Section

■ ■ 14. Hands-On Coding Section

14.1 Complete E2E Login Test

```
// tests/e2e/login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../../pages/LoginPage';
import { DashboardPage } from '../../pages/DashboardPage';

test.describe('Authentication E2E', () => {
  let loginPage: LoginPage;
  let dashboard: DashboardPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    dashboard = new DashboardPage(page);
    await loginPage.navigate();
  });

  test('TC001 - Successful login', async ({ page }) => {
    await test.step('Enter credentials', async () => {
      await loginPage.login('admin@test.com', 'Admin@123');
    });
    await test.step('Verify dashboard', async () => {
      await expect(page).toHaveURL(/dashboard/);
      await expect(dashboard.welcomeMsg).toContainText('Welcome');
    });
  });

  test('TC002 - Empty fields validation', async () => {
    await loginPage.submitBtn.click();
    await expect(loginPage.emailError).toHaveText('Email is required');
    await expect(loginPage.passError).toHaveText('Password is required');
  });

  test('TC003 - Logout redirects to login', async ({ page }) => {
    await loginPage.login('admin@test.com', 'Admin@123');
    await dashboard.logout();
    await expect(page).toHaveURL('/login');
  });
});
```

14.2 POM Framework Example

```
// pages/BasePage.ts
import { Page } from '@playwright/test';
export abstract class BasePage {
  constructor(protected page: Page) {}
```

```
async waitForLoad() {
  await this.page.waitForLoadState('networkidle');
}

// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';
import { BasePage } from './BasePage';

export class LoginPage extends BasePage {
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly submitBtn: Locator;
  readonly errorMsg: Locator;
  readonly emailError: Locator;
  readonly passError: Locator;

  constructor(page: Page) {
    super(page);
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.submitBtn = page.getByRole('button', { name: 'Sign In' });
    this.errorMsg = page.locator('.error-message');
    this.emailError = page.locator('.email-error');
    this.passError = page.locator('.pass-error');
  }

  async navigate() { await this.page.goto('/login'); }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitBtn.click();
  }
}
```

14.3 API + UI Combined Test

```
// tests/combined/user-management.spec.ts
import { test, expect } from '@playwright/test';

test('create user via API, verify in UI, delete via API', async ({ page, request }) => {
  // Step 1: Create test user via API
  const createResp = await request.post('/api/users', {
    data: { name: 'Test User', email: `test.${Date.now()}@example.com`, role: 'viewer' },
    headers: { Authorization: `Bearer ${process.env.AUTH_TOKEN}` },
  });
  expect(createResp.status()).toBe(201);
  const { id, email } = await createResp.json();

  // Step 2: Verify user appears in UI
  await page.goto('/admin/users');
  await page.getByPlaceholder('Search users').fill(email);
```

```
await expect(page.locator('table tr').filter({ hasText: email })).toBeVisible();

// Step 3: Cleanup via API
const delResp = await request.delete(`/api/users/${id}`, {
  headers: { Authorization: `Bearer ${process.env.AUTH_TOKEN}` },
});
expect(delResp.status()).toBe(204);
});
```

14.4 Complete Mocking Example

```
// tests/mocking/dashboard-mock.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Dashboard with mocked data', () => {
  test.beforeEach(async ({ page }) => {
    await page.route('**/api/stats', (r) => r.fulfill({
      contentType: 'application/json',
      body: JSON.stringify({ users: 1250, revenue: 45600, orders: 89 }),
    }));
    await page.route('**/api/notifications', (r) => r.fulfill({
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, msg: 'New order #1001', type: 'info' },
        { id: 2, msg: 'Low stock alert', type: 'warning' },
      ]),
    }));
  });

  test('stats display correctly', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page.locator('[data-stat="users"]')).toHaveText('1,250');
    await expect(page.locator('[data-stat="revenue"]')).toHaveText('$45,600');
  });

  test('2 notifications shown', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page.locator('.notification')).toHaveCount(2);
    await expect(page.locator('.notification').first()).toContainText('New order');
  });
});
```

SECTION 15 — Quick Revision Cheat Sheet

■ 15. Quick Revision Cheat Sheet

NAVIGATION	LOCATORS (Best First)
<code>page.goto(url) / page.goto('/path')</code> (uses baseUrl)	<code>getByRole('button', { name: 'Submit' })</code>
<code>page.reload() / page.goBack() / page.goForward()</code>	<code>getByLabel('Email') / getByPlaceholder('Enter email')</code>
<code>page.waitForURL('**/dashboard')</code>	<code>getByText('Welcome') / getByTestId('my-id')</code>
<code>page.waitForLoadState('networkidle')</code>	<code>locator('.css-class') / locator('#id')</code>
<code>expect(page).toHaveURL(pattern)</code>	<code>locator('tr').filter({ hasText: 'John' })</code>
<code>expect(page).toHaveTitle('Page Title')</code>	<code>locator('.item').nth(2) / .first() / .last()</code>
ACTIONS	ASSERTIONS
<code>locator.click() / .dblclick() / .hover()</code>	<code>expect(loc).toBeVisible() / .toBeHidden()</code>
<code>locator.fill('value') / .clear() / .type('text')</code>	<code>expect(loc).toHaveText('str') / .toContainText('str')</code>
<code>locator.check() / .uncheck() / .selectOption('val')</code>	<code>expect(loc).toHaveValue('val') / .toHaveCount(n)</code>
<code>locator.dragTo(targetLocator)</code>	<code>expect(loc).toBeEnabled() / .toBeDisabled()</code>
<code>page.keyboard.press('Enter') / 'Control+A'</code>	<code>expect(loc).toHaveAttribute('attr', 'val')</code>
<code>input[type=file].setInputFiles('path/to/file')</code>	<code>expect.soft(loc).toHaveText('val') // non-blocking</code>
WAITS (Smart)	NETWORK MOCKING
<code>expect(locator).toBeVisible() – PREFERRED</code>	<code>page.route('**/api/users', route => {})</code>
<code>page.waitForResponse('**/api/endpoint')</code>	<code>route.fulfill({ status, contentType, body })</code>
<code>page.waitForEvent('download' 'page')</code>	<code>route.continue({ headers: {...} })</code>
<code>page.waitForLoadState('networkidle')</code>	<code>route.abort() – block the request</code>
<code>expect.poll(() => fn(), { timeout }).toBe(val)</code>	<code>page.unroute('**/api/users')</code>
<code>page.waitForTimeout(ms) – AVOID!</code>	<code>page.route('**/*.png', r => r.abort()) // block images</code>
API TESTING	CONFIG KEYS
<code>request.get('/api/users')</code>	<code>fullyParallel: true / workers: 4 or '50%'</code>
<code>request.post('/api/users', { data: {...} })</code>	<code>retries: 2 / timeout: 30000</code>
<code>request.put('/api/users/1', { data: {...} })</code>	<code>use: { baseUrl, headless, viewport }</code>
<code>request.delete('/api/users/1')</code>	<code>use: { trace, video, screenshot }</code>
<code>expect(resp.status()).toBe(200)</code>	<code>reporter: [['html'], ['json']]</code>
<code>const body = await resp.json()</code>	<code>projects: [{ name, use: {...devices['...']} }]</code>

HOOKS & FIXTURES`test.beforeAll / test.afterAll``test.beforeEach / test.afterEach``test.extend({ fixture: async ({page}, use) => {} })``test.step('Step name', async () => {})``testInfo.retry / testInfo.status / .title``await testInfo.attach('name', { path, contentType })`**CLI COMMANDS**`npx playwright test --run-all``npx playwright test --grep @smoke --by tag``npx playwright test --debug --debugger``npx playwright test --headed --visible-browser``npx playwright codegen https://url --record``npx playwright show-report --open HTML report`

SECTION 16 — Bonus Section

■ 16. Bonus Section

16.1 Common Mistakes in Playwright

Mistake	Better Approach
Using <code>waitForTimeout()</code>	-> Replace with <code>expect().toBeVisible()</code> or <code>waitForResponse()</code>
Hardcoding URLs in tests	-> Always configure <code>baseUrl</code> and use relative paths
No test isolation	-> Each test must set up and tear down its own state
Using <code>page.\$()</code> / <code>page.\$()</code>	-> Use <code>page.locator()</code> — <code>ElementHandle</code> is legacy API
No POM structure	-> Without POM, a single UI change breaks many test files
Re-logging in every test	-> Use <code>storageState</code> to reuse sessions — saves time
Not using fixtures	-> Fixtures eliminate code duplication in test setup
Hard assertions everywhere	-> Use <code>expect.soft()</code> when checking multiple UI elements
No test tagging	-> Tag tests <code>@smoke @regression</code> for selective CI runs
Index-based locators	-> Avoid <code>.nth(0)</code> for single items — use specific locators

16.2 Interview Tips for SDET Roles

- Always explain **WHY** you chose a solution, not just what you did
- Mention **trade-offs**: speed vs stability, POM complexity vs simplicity
- Show CI/CD integration knowledge — interviewers love this
- Discuss test pyramid: unit (70%) → integration (20%) → E2E (10%)
- Talk about **metrics**: execution time, pass rate, flakiness percentage
- Mention Playwright's trace viewer — shows deep debugging knowledge
- Explain how you handle test data — API setup vs UI setup
- Be ready to **write code** on whiteboard or shared screen
- Know validation vs verification in QA context
- Discuss how you prioritize which tests to automate first
- Show knowledge of parallel execution and sharding strategies
- Mention reporting tools: Allure, HTML report, custom reporters

16.3 Resume Tips for Automation Engineers

- **Quantify impact**: 'Reduced regression suite time from 4hrs to 45min using parallel execution'
- **List technologies**: Playwright, TypeScript, GitHub Actions, Allure, Docker, Jenkins
- **Coverage metrics**: 'Maintained 800+ automated tests across 3 environments'
- **Architecture decisions**: 'Designed POM framework adopted by 5-member QA team'
- **API testing**: 'Implemented API contract tests using Playwright + AJV schema validation'
- **CI/CD**: 'Integrated automation in GitHub Actions with Slack failure notifications'
- **Action verbs**: Designed, Built, Optimized, Reduced, Increased, Led, Mentored